

Similarity Metrics Explained Visually

Cosine, Euclidean, Manhattan, dot product. Same two vectors, four different answers about how close they are.





Turning "Are These Alike" Into A Number

Every embedding is a point. A metric measures the gap.

📍 THE SETUP

"refund policy" → [0.21, -0.44, 0.87, ...]

"return window" → [0.19, -0.41, 0.85, ...]

"shipping cost" → [-0.62, 0.33, -0.11, ...]

Two of these sit close together. One does not.

↑ SIMILARITY

Higher means **more** alike. Cosine and dot product. You sort **descending**.

↓ DISTANCE

Higher means **less** alike. Euclidean and Manhattan. You sort **ascending**.



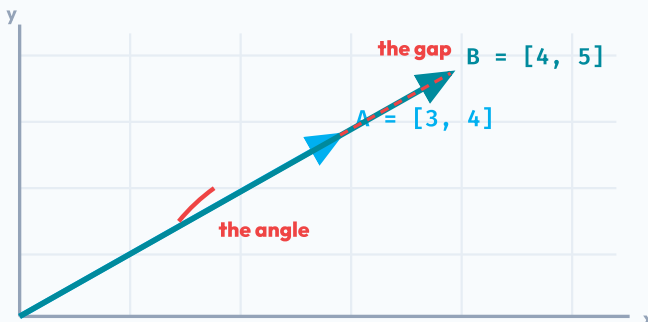
Getting the sort direction backwards is a real bug that produces **plausible looking, completely wrong results**.



Direction And Length

Two vectors give you two different things to measure

TWO VECTORS ON A PLANE



< THE ANGLE

Small angle means pointing the same way, which for text means similar **meaning**. This is what **cosine** measures.

THE GAP

Short gap means the points sit near each other in the space. This is what **euclidean** measures.

💡 For text embeddings, **direction carries the meaning** and length usually carries something else, like document length. That one fact explains most of this deck.



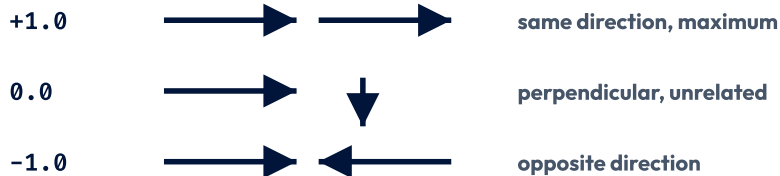
Cosine Similarity

Measure the angle, ignore the length

$$\cos(A, B) = A \cdot B / (\|A\| \times \|B\|)$$

Dot product divided by both lengths. That division is what removes magnitude from the answer.

↔ THE RANGE



For most text models, values sit in a narrow band well above zero. Treat 0.0 as unrelated rather than expecting to see it.

= THE DEFINING PROPERTY

A = [1, 2, 3]

B = [10, 20, 30] ten times longer, same direction

$\cosine(A, B) = 1.0$

Length is discarded completely. A one sentence answer and a five paragraph answer about the same topic score the same.

PYTHON

```
def cosine_similarity(a, b):  
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```



Magnitude Is Usually Noise, Not Signal

The problem cosine quietly solves for you

📄 TWO DOCUMENTS, ONE QUERY

DOC 1

"The refund window is 30 days."

SHORT

DOC 2

The same fact, plus four more paragraphs repeating the word refund

LONG

In many embedding spaces doc 2 produces a larger magnitude, simply because there is more of it.

✗ WITH DOT PRODUCT

Doc 2 wins because it is bigger, even though doc 1 answers the question more directly.

✓ WITH COSINE

Both are judged on direction alone, so relevance decides rather than length.



Use cosine when you want "is this about the same thing", and you do not want longer documents winning automatically.





Dot Product, Direction And Size

Cosine without the division

$$A \cdot B = a_1 \cdot b_1 + a_2 \cdot b_2 + a_3 \cdot b_3 + \dots$$

Multiply matching positions and add. No division, so no normalisation.

🔗 THE RELATIONSHIP TO COSINE

$$\text{dot}(A, B) = \underset{\text{direction}}{\text{cosine}(A, B)} \times \underset{\text{magnitudes}}{\|A\| \times \|B\|}$$

Cosine is the dot product with magnitude divided out. If every vector has length 1, both multipliers become 1 and the two are the same number.

⚡ **Normalise once at index time, then use dot product.** You get cosine with fewer operations per query, and databases can use their fastest index types.

PYTHON

```
# Normalise once, at index time
vectors = model.encode(texts, normalize_embeddings=True)

index = faiss.IndexFlatIP(768) # IP = inner product
index.add(vectors)

# Everything is normalised, so these scores ARE cosine
query = model.encode([q], normalize_embeddings=True)
scores, ids = index.search(query, k=5)
```





When Magnitude Is Carrying Information

Three cases where you want size to count

RECOMMENDERS

User vector length can encode engagement, item length can encode popularity. A popular item **should** rank higher, all else equal.

IP TRAINED MODELS

Some retrieval models are trained with an inner product objective and expect unnormalised vectors. Normalising anyway can measurably hurt recall.

LEARNED SCORING

When magnitude is a deliberate output of training rather than an artefact of document length

⚠ THE PRACTICAL WARNING

Check what your embedding model recommends. Most modern text models are trained for cosine and say so in their documentation.

Using dot product on unnormalised vectors from a cosine trained model gives you **a length bias you did not ask for**, and it will not raise an error.



The model card outranks the habit. **Read it before you choose.**



Euclidean, The Straight Line

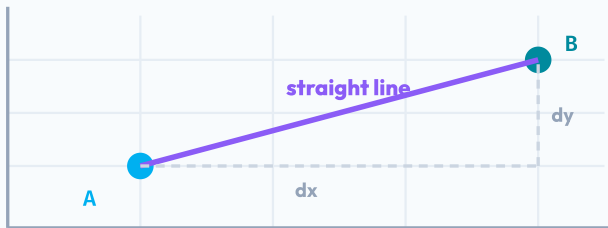
Pythagoras, extended to any number of dimensions

$$L2(A, B) = \text{sqrt}((a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots)$$

Square every difference, add them up, take the root.



AS THE CROW FLIES



0

Identical points

SMALL

Close together

LARGE

Far apart, and unbounded above



There is no maximum, so raw values are hard to read. Is 4.7 close? It depends entirely on your data.





On Normalised Vectors, They Rank The Same

The scores differ. The order does not.

$$L2(A, B)^2 = 2 - 2 \times \cos(A, B)$$

For unit length vectors. As cosine goes up, euclidean goes down, in a fixed relationship.

COSINE	EUCLIDEAN
1.00	0.00
0.90	0.45
0.50	1.00
0.00	1.41



Sorting by one is the same as sorting by the other, reversed.

WHERE THEY GENUINELY DIVERGE

$A = [1, 1]$ cosine = 1.00 identical
 $B = [100, 100]$ euclidean = 140.0 enormously far

On unnormalised vectors they disagree completely. So "cosine or euclidean" is often really "are my vectors normalised".



Manhattan, Along The Grid

You cannot cut diagonally through buildings

$$L1(A, B) = |a_1 - b_1| + |a_2 - b_2| + \dots$$

Sum of absolute differences. No squaring, no square root.



WALKING THE STREETS



euclidean 4.24

manhattan 6



WHERE THEY DISAGREE

A = [0, 0, 0, 0]

B = [10, 0, 0, 0] one big difference

C = [3, 3, 3, 3] four small ones

euclidean: A to B = 10.0 A to C = 6.0 B is further

manhattan: A to B = 10.0 A to C = 12.0 C is further

Euclidean **squares** each difference, so one large gap dominates. Manhattan sums linearly, so many small gaps count as much as one big one.



When Manhattan Is Worth Considering

Rarely the default for text, useful in specific cases

OUTLIERS

No squaring, so one extreme dimension has less power to dominate the score

SPARSE VECTORS

When most dimensions are zero, linear summing can behave more sensibly than squaring

REAL UNITS

When dimensions are price, weight or count, "total difference across attributes" is a meaningful number

HIGH DIMENSIONS

Some analyses suggest L1 degrades more gracefully than L2 as dimensionality grows, though this is data dependent



THE HONEST SUMMARY FOR TEXT

Dense text embeddings are trained with cosine or inner product objectives. Manhattan is not what the model learned to optimise, and not what your vector database is tuned for.



Know what it is. **Reach for cosine anyway**, unless you have measured something else working better on your data.



Same Vectors, Every Metric

A = [3, 4] B = [4, 3] C = [30, 40]

METRIC	A TO B	A TO C	READING
COSINE	0.960	1.000	C is a scaled A, so identical direction
DOT	24.0	250.0	C wins by magnitude alone, not relevance
EUCLIDEAN	1.41	27.0	C is geometrically far despite pointing the same way
MANHATTAN	2.00	36.0	Same conclusion as euclidean, larger number

? THE DISAGREEMENT IN ONE LINE

Is C the SAME as A, or FAR from A?

COSINE SAYS identical

EUCLIDEAN SAYS very far

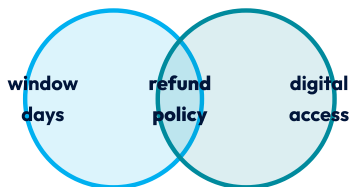
== Both are correct. They are answering different questions.



When Vectors Are Not Dense Embeddings

Jaccard for sets, Hamming for bits

🔗 JACCARD · OVERLAP OVER UNION



intersection = 2
union = 6
 $J = 0.33$

Used for deduplication, near duplicate detection, tag overlap and MinHash shingles.

🔗 HAMMING · COUNT THE DIFFERING BITS

A = 1 0 1 1 0 1 0 0
B = 1 0 0 1 0 1 1 0
= = X = = = X =

2 differences, Hamming = 2

On modern hardware this is an XOR followed by a bit count, **which is why binary embeddings search so quickly.**

🔗 Match the metric to the **shape of the data**, not to what you used last time.



The One That Predates Embeddings

Not geometric at all, and still essential

TERM FREQUENCY

How often the query word appears here, with diminishing returns, so 50 mentions is not 50 times better than one

INVERSE DOC FREQ

How rare the word is across the corpus, so "the" scores near zero and "photosynthesis" scores high

LENGTH NORM

So long documents do not win just by containing more words

🔑 WHY IT STILL MATTERS

It matches **exact strings**. Product codes, error codes, proper nouns and rare identifiers are precisely where dense embeddings are weakest.

dense "how do I get my money back" → "refund policy"

sparse "SKU-4471" → "SKU-4471"

$$\text{score}(\text{doc}) = \text{sum over lists of } 1 / (k + \text{rank})$$

Reciprocal Rank Fusion combines two ranked lists without their scores being on the same scale. A document ranked highly by either method rises.





Choosing A Metric

Start with the shape of your vectors

DENSE TEXT

Check the model card. It almost always says cosine. **Normalise at index time, use inner product.**

MODEL SAYS IP

If the documentation specifically says dot product on unnormalised vectors, follow the model, not the habit

IMAGE, MULTIMODAL

Usually cosine, for the same reason

BINARY, QUANTIZED

Hamming distance, for speed

SETS, TAGS

Jaccard

RAW FEATURES

Euclidean, or Manhattan if outlier robustness matters. Scale the features first.

KEYWORD TEXT

BM25, combined with dense search through RRF



For dense text: normalise, use inner product, and spend the tuning effort elsewhere. Metric choice is rarely where the recall is hiding.





Normalisation, Practically

The one step that determines most of this deck

PYTHON

```
def normalise(vectors):  
    """Scale every vector to length 1, preserving direction."""  
    norms = np.linalg.norm(vectors, axis=1, keepdims=True)  
    return vectors / np.clip(norms, 1e-12, None)  
  
# Most libraries do it for you  
vectors = model.encode(texts, normalize_embeddings=True)  
  
# Verify it actually happened  
lengths = np.linalg.norm(vectors, axis=1)  
assert np.allclose(lengths, 1.0), f"not normalised: {lengths[:5]}"
```

WHAT IT DOES

before	[3, 4]	length 5
after	[0.6, 0.8]	length 1

⚠ Normalise at index time and query time, or at neither. Mixing them gives scores that are neither cosine nor dot product, with no error raised.

🔪 THE OTHER RULE

Scale features before using euclidean or manhattan on raw numeric data. With age spanning 50 and income spanning 180,000, **income differences swamp age entirely**. Text embeddings avoid this because every dimension comes from the same learned space.





What Goes Wrong With Metrics

Most of these fail silently

× SETUP

sorting a distance as if it were a similarity
normalising at index but not at query
dot product on unnormalised cosine vectors
picking a metric by habit, not by model card
euclidean on unscaled raw features

× INTERPRETATION

comparing raw scores across different models
assuming a threshold transfers between
models
dense only, so exact codes never match
tuning the metric when chunking is the
problem

SYMPTOM

LIKELY CAUSE

"results are the opposite"

distance sorted as a similarity

"long documents always win"

dot product on unnormalised vectors

"scores changed after upgrade"

new model, new distribution, thresholds do not
transfer

"exact codes never match"

dense only, no BM25 in the mix

"everything scores near 0.8"

normal for text. Look at rank, not absolute value.



Direction Or Distance, And Whether Size Counts

Every metric here answers one of two questions

METRIC	WHAT IT MEASURES	WHERE IT BELONGS
COSINE	angle only, length ignored	the text default
DOT	angle and length together	equals cosine when normalised
EUCLIDEAN	straight line gap	same ranking as cosine, normalised
MANHATTAN	grid walk, linear sums	outlier robust, rare for text
JACCARD	set overlap	dedup, tags, shingles
HAMMING	differing bits	binary vectors, very fast
BM25	keyword statistics	exact terms, hybrid search

☰ THREE RULES THAT PREVENT MOST PROBLEMS

- 1 Normalise at index **and** query, or at neither
- 2 Read the model card before choosing
- 3 Compare ranks, not raw scores across models



Chunking and the embedding model move recall far more than this ever will.
Follow AI Coach John for more.



AI Coach John

Follow

PROITBRIDGE

Repost